

TÀI LIỆU ÔN TẬP BẢO VỆ ĐỒ ÁN

Tổng hợp các câu hỏi trọng tâm về Kiến Trúc Hệ Thống, Docker & CI/CD

PHẦN 1: KIẾN TRÚC TRIỂN KHAI & MÔI TRƯỜNG

Q: Kiến trúc triển khai của dự án em là gì? Em phân bố các thành phần như thế nào?

A: Hệ thống của em sử dụng kiến trúc phân tán (Client-Server), tách biệt hoàn toàn giữa Frontend (FE) và Backend (BE).

- **Frontend (Giao diện người dùng):** Chứa các file tĩnh HTML/CSS/JS, được lưu trữ và phục vụ bởi nền tảng đám mây Netlify.

- **Backend & Database:** Ứng dụng Java Spring Boot và Cơ sở dữ liệu MySQL 8.0 được triển khai trên một máy chủ ảo (VPS) chạy hệ điều hành Ubuntu. Hai thành phần này giao tiếp với nhau trong môi trường nội bộ của VPS thông qua Docker.

Q: Tại sao em lại sử dụng Docker để chạy Backend và Database thay vì cài trực tiếp phần mềm lên VPS?

A: Em sử dụng Docker để giải quyết bài toán "Works on my machine" (Chạy được trên máy em nhưng lên server thì lỗi). Docker giúp đóng gói ứng dụng (Java Spring Boot) và CSDL (MySQL) cùng với toàn bộ môi trường thực thi của chúng thành các Container độc lập. Điều này đảm bảo tính nhất quán 100% giữa lúc dev trên máy cá nhân và lúc chạy trên VPS, đồng thời giúp em dễ dàng khởi động lại toàn bộ hệ thống chỉ bằng một câu lệnh `docker compose up` mà không sợ xung đột phiên bản hệ điều hành.

PHẦN 2: CẤU HÌNH BẢO MẬT & KẾT NỐI MẠNG

Q: Frontend của em chạy trên Netlify (HTTPS), còn Backend chạy trên VPS (HTTP). Làm sao em giải quyết được lỗi chặn bảo mật Mixed Content và CORS?

A: Để giải quyết triệt để lỗi này, em sử dụng kỹ thuật Proxy (Chuyển tiếp). Trên Netlify, em cấu hình một file `_redirects` với luật chuyển tiếp mọi request có tiền tố `/api/*` về địa chỉ IP và port 8000 của VPS. Khi đó, Netlify đóng vai trò là một máy chủ trung gian: Trình duyệt của người dùng chỉ giao tiếp với Netlify qua HTTPS, sau đó Netlify tự động gọi ngầm về Backend HTTP của em. Cách này vừa vượt qua được lỗi CORS, vừa ẩn được địa chỉ IP thật của máy chủ VPS.

Q: Em đã cấu hình tường lửa (Firewall) cho máy chủ VPS như thế nào để đảm bảo an toàn?

A: Em sử dụng UFW (Uncomplicated Firewall) trên Ubuntu kết hợp với Security Group của nhà cung cấp Cloud. Mặc định em đóng toàn bộ các cổng truy cập từ bên ngoài, chỉ mở đúng 3 cổng thiết yếu:

- **Cổng 8686 (TCP):** Dành riêng cho giao thức SSH để em quản trị server.
- **Cổng 8000 (TCP):** Mở cho Backend Java Spring Boot để Netlify có thể lấy dữ liệu.
- **Cổng 3306 (TCP):** Mở cho MySQL để em có thể quản trị cơ sở dữ liệu từ xa khi cần.

PHẦN 3: TỰ ĐỘNG HÓA CI/CD & KIỂM THỬ

Q: Quy trình đưa code từ máy cá nhân lên server (CI/CD Pipeline) của em hoạt động như thế nào?

A: Em sử dụng GitHub Actions làm trung tâm điều phối. Luồng xử lý được chia làm hai phần:

- **Đối với Frontend:** Khi có code đẩy lên nhánh main, GitHub Actions sẽ kích hoạt chạy kiểm thử giao diện tự động (End-to-End Test) bằng công cụ Cypress. Nếu toàn bộ test case đều pass (hiển thị xanh), mã nguồn mới sẽ được Netlify tự động kéo về và triển khai lên mạng.
- **Đối với Backend:** Em cài đặt một Self-hosted Runner trực tiếp trên VPS. Khi có code mới, GitHub sẽ ra lệnh cho Runner này tự động dừng các Docker container cũ, lấy code mới về, đóng gói lại (build) và khởi chạy container mới thông qua Docker Compose mà không cần em phải thao tác tay.

Q: Tại sao em lại phải đặt Job chạy Cypress trước khi Deploy? Nếu bỏ qua bước này có sao không?

A: Việc đặt Cypress kiểm thử trước khi Deploy là tiêu chuẩn bắt buộc của quy trình Continuous Integration (Tích hợp liên tục). Việc này tạo ra một "chiếc khiên" bảo vệ. Nếu code mới có lỗi (ví dụ nút đăng nhập bị hỏng), Cypress sẽ phát hiện và đánh dấu thất bại (Fail) ngay lập tức. Quá trình Deploy sẽ bị chặn lại, nhờ đó phiên bản lỗi sẽ không bao giờ bị đẩy lên hệ thống thật cho người dùng cuối nhìn thấy.